

REST API

REST(Representational State Transfer) আসলে কী?

REST কোনো programming language না, framework না।

এটা একটা **architectural style** — মানে API বানানোর নিয়মকানুন।

👉 REST বলে দেয়:

- URL কেমন হবে
- কোন HTTP method কী কাজ করবে
- Data কিভাবে আদান-প্রদান হবে

REST API দিয়ে কী হয়?

REST API দিয়ে:

- Frontend ↔ Backend কথা বলে
- Mobile App ↔ Server কথা বলে
- Microservices একে-অপরের সাথে কথা বলে

📌 উদাহরণ:

- Facebook App → Facebook Server
- React Frontend → Django / Node Backend

সবই REST API দিয়ে।

REST API এর Real Life Example 🍔

ধরো তুমি **Food Delivery App** বানাচ্ছে।

API Endpoints

GET	/foods	→ সব খাবার দেখাও
GET	/foods/5	→ id=5 খাবার দেখাও
POST	/foods	→ নতুন খাবার যোগ করো
PUT	/foods/5	→ খাবার আপডেট করো
DELETE	/foods/5	→ খাবার ডিলিট করো

HTTP Methods (খুব গুরুত্বপূর্ণ) ⚡

Method	কাজ
GET	Data আনতে
POST	নতুন Data তৈরি
PUT	পুরো Data আপডেট
PATCH	আংশিক আপডেট
DELETE	Data ডিলিট

👉 REST API এই methods গুলো standard ভাবে ব্যবহার করে।

REST API কিভাবে কাজ করে? (Flow)

Client (React / App)

|

| HTTP Request

↓

REST API Server (Django / Node)

|

| JSON Response

↓

Client

📌 Request:

GET /users/1

📌 Response:

```
{  
  "id": 1,  
  "name": "Arman",  
  "email": "arman@gmail.com"  
}
```

REST API এর Core Principles (Interview Favorite ★)

Stateless

Server কোনো user এর previous request মনে রাখে না
→ প্রতিটা request আলাদা

✗ Session based না

✓ Token (JWT) based authentication

Client-Server Separation

Frontend আর Backend আলাদা

→ scalable & maintainable

One-Line Definition

REST API is an architectural style that uses HTTP methods to perform CRUD operations on resources in a stateless manner.

১. REST কী?

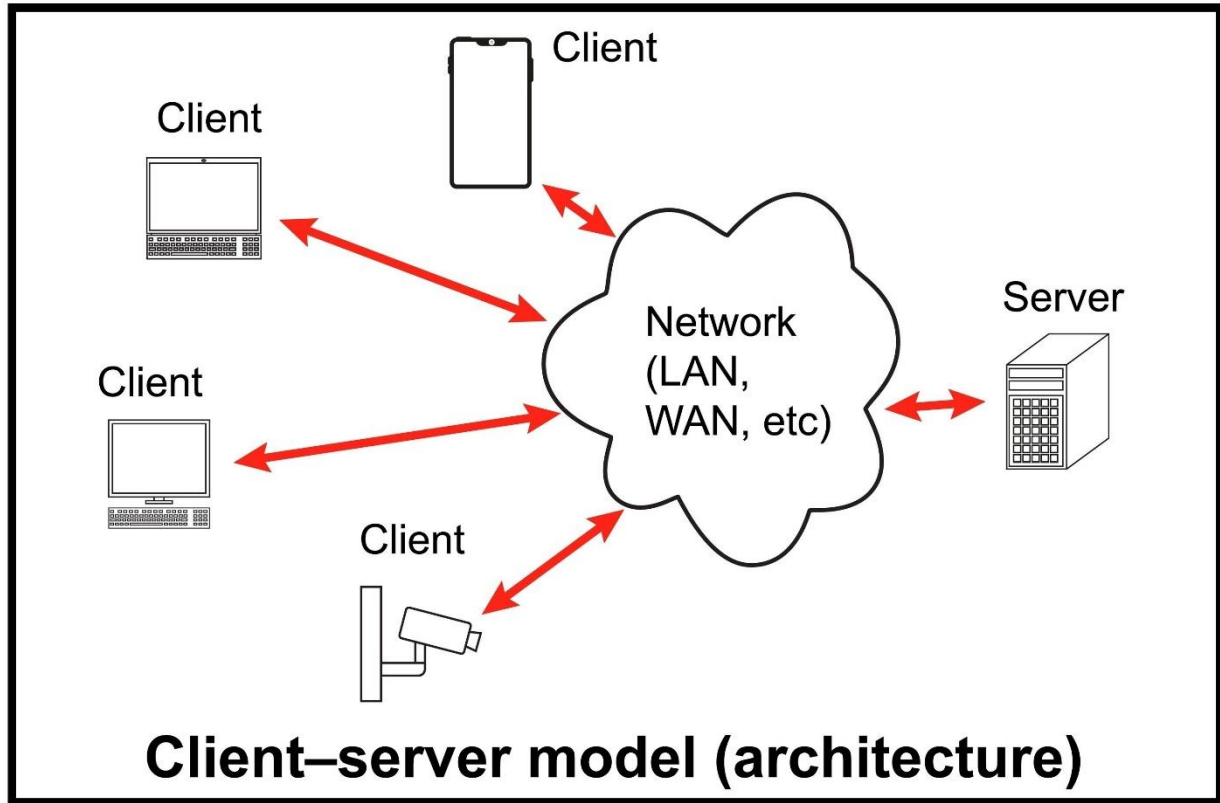
REST-এর পূর্ণরূপ হলো **Representational State Transfer**। এটি কোনো প্রোগ্রামিং ল্যাঙ্গুয়েজ নয়, বরং এটি একটি **Architectural Style** বা ডিজাইনের নিয়ম। যখন কোনো ওয়েব সার্ভিস এই নিয়মগুলো মেনে চলে, তখন তাকে **RESTful API** বলা হয়।

২. এটি কীভাবে কাজ করে? (Client-Server Model)

REST API মূলত **Client** এবং **Server**-এর মধ্যে কাজ করে।

- **Client:** যে তথ্য চায় (যেমন: আপনার মোবাইল অ্যাপ বা ব্রাউজার)।
- **Server:** যার কাছে তথ্য জমা থাকে (Database)।

ক্লায়েন্ট যখন সার্ভারের কাছে কোনো রিকোয়েস্ট পাঠায়, সার্ভার তখন সেই রিকোয়েস্ট প্রসেস করে এবং একটি রেসপন্স (Response) পাঠায়। এই আদান-প্রদান সাধারণত **JSON** (JavaScript Object Notation) ফরম্যাটে হয়, যা মানুষ এবং মেশিন উভয়েই সহজে পড়তে পারে।



৩. REST API-এর মূল উপাদানসমূহ

একটি REST API রিকোয়েস্টে সাধারণত ৪টি প্রধান অংশ থাকে:

1. **Endpoint (URL):** এটি হলো সার্ভারের ঠিকানা যেখানে রিকোয়েস্ট পাঠানো হয়। যেমন: <https://api.example.com/users>
2. **HTTP Methods:** আপনি কী করতে চান তা এই মেথড দিয়ে বোঝানো হয়।
3. **Headers:** এখানে রিকোয়েস্ট সম্পর্কে অতিরিক্ত তথ্য থাকে (যেমন: মেটাডেটা, অথেনটিকেশন টোকেন)।
4. **Body/Payload:** যদি সার্ভারে কোনো তথ্য পাঠাতে হয় (যেমন: নতুন ইউজার তৈরি করা), তবে তা বডিতে থাকে।

৪. প্রধান HTTP Methods (ক্রিয়া)

REST API মূলত ৪টি প্রধান কাজ করে, যেগুলোকে সংক্ষেপে **CRUD** বলা হয়:

Method	কাজ (Action)	উদাহরণ
GET	তথ্য পড়া (Read)	সার্ভার থেকে ইউজার লিস্ট নিয়ে আসা।
POST	নতুন তথ্য তৈরি করা (Create)	নতুন একটি অ্যাকাউন্ট খোলা।
PUT	তথ্য আপডেট করা (Update)	প্রোফাইল পিকচার বা নাম পরিবর্তন করা।
DELETE	তথ্য মুছে ফেলা (Delete)	কোনো পোস্ট বা অ্যাকাউন্ট ডিলিট করা।

৫. REST-এর কিছু গুরুত্বপূর্ণ বৈশিষ্ট্য (Principles)

একটি এপিআই-কে তখনই "RESTful" বলা যাবে যখন সে নিচের শর্তগুলো পূরণ করবে:

- **Statelessness:** সার্ভার ক্লায়েন্টের আগের কোনো তথ্য মনে রাখবে না। প্রতিটি রিকোয়েস্ট হবে নতুন এবং এতে প্রয়োজনীয় সব তথ্য থাকতে হবে।
- **Uniform Interface:** সব রিকোয়েস্টের ধরন এবং ফরম্যাট একই রকম হতে হবে যাতে ব্যবহার করা সহজ হয়।
- **Cacheable:** ক্লায়েন্ট চাইলে কিছু রেসপন্স সেভ (Cache) করে রাখতে পারে যাতে বারবার লোড হতে সময় না লাগে।
- **Client-Server Independence:** ক্লায়েন্ট এবং সার্ভার আলাদাভাবে কাজ করবে। সার্ভারে কোনো পরিবর্তন করলে ক্লায়েন্টের কোডে সমস্যা হবে না।

৬. একটি বাস্তব উদাহরণ

মনে করুন, আপনি একটি অ্যাপে লগইন করছেন:

1. আপনার অ্যাপ (Client) সার্ভারের কাছে একটি **POST** রিকোয়েস্ট পাঠাবে
api.example.com/login এই এন্ডপয়েন্টে।
2. সাথে আপনার ইউজারনেম এবং পাসওয়ার্ড পাঠাবে।
3. সার্ভার আপনার তথ্য চেক করবে।
4. যদি সব ঠিক থাকে, সার্ভার একটি **JSON** রেসপন্স পাঠাবে: {"status":
"success", "token": "xyz123"}।